

Build Your Own Kafka

Propuesta de proyecto — Sistemas Distribuidos

Modalidad: individual | Metodología: Kanban | Repositorio: github.com/Chillipeeper1/build-your-kafka

1. Objetivo

Objetivo general

Diseñar e implementar, en Java, un sistema de mensajería distribuido simplificado inspirado en Apache Kafka, replicando sus mecanismos centrales de coordinación, particionamiento, replicación y tolerancia a fallos, como vehículo práctico para el aprendizaje de sistemas distribuidos.

Objetivos específicos

1. Implementar un cliente de coordinación sobre Apache ZooKeeper (sesión asíncrona, nodos persistentes/efímeros, watches reactivos, manejo de expiración de sesión).
2. Implementar el broker central (`SimpleKafkaBroker`): registro en el cluster, elección de controller, comunicación de red, y manejo de producción/consumo/replicación de mensajes.
3. Construir la capa de almacenamiento (`Partition`): log segmentado con índice para persistencia eficiente en disco.
4. Desarrollar una librería cliente (`SimpleKafkaClient`) que abstraiga el descubrimiento de brokers y el enrutamiento por partición.
5. Construir APIs de alto nivel — `SimpleKafkaProducer` y `SimpleKafkaConsumer` — con manejo de offsets y polling continuo.
6. Validar el sistema en un escenario multi-broker real, incluyendo caída y recuperación de nodos.

2. Justificación

Un sistema de mensajería distribuido como Kafka concentra, en un solo proyecto, prácticamente todos los problemas centrales de la materia: coordinación entre nodos sin un reloj global compartido, elección de líder, detección de fallos, replicación de datos, tolerancia a particiones de red, y el tradeoff entre consistencia y disponibilidad. Construirlo desde cero — en vez de solo usarlo como caja negra — obliga a resolver esos problemas de forma explícita en vez de darlos por sentado.

Esto ya se evidenció en la primera etapa del proyecto: al implementar la elección de controller, surgió un conflicto real entre dos decisiones de diseño (un nodo `/controller` creado de antemano

como persistente rompía la elección, que necesita crearlo de forma efímera) — un ejemplo concreto de por qué construir el mecanismo en persona deja una comprensión que leer sobre él no deja.

El proyecto además ya estaba en marcha como iniciativa personal antes del requerimiento de la cátedra, lo cual juega a favor: hay interés genuino sostenido, no una tarea armada a último momento solo para cumplir el requisito.

3. Análisis de requisitos

Requisitos funcionales

#	Requisito	Estado
RF1	El sistema debe permitir que un broker se conecte y coordine su estado vía ZooKeeper (sesión, registro, watches).	Implementado y verificado
RF2	El sistema debe elegir automáticamente un único controller entre varios brokers, y reelegir si el actual falla.	Implementado y verificado
RF3	Persistir mensajes en disco mediante segmentos de log (chunks de 1MB) con archivos de índice para mapeo eficiente offset→posición, en una clase <code>Partition</code> con soporte de replicación líder-seguidor y control de concurrencia vía <code>ReadWriteLock</code> (múltiples lectores, un escritor).	Backlog
RF4	El broker debe aceptar conexiones de red y procesar operaciones de producir/consumir/replicar mensajes.	Backlog
RF5	Debe existir una librería cliente que descubra brokers y enrute mensajes a la partición correcta sin exponer el protocolo de red al usuario.	Backlog
RF6	Debe existir una API de productor y una de consumidor, con manejo de offsets y serialización.	Backlog
RF7	El sistema debe demostrarse en un cluster multi-broker real, incluyendo tolerancia a fallos (caída/recuperación de un nodo).	Backlog

Requisitos no funcionales

- **Corrección por sobre rendimiento:** el objetivo es demostrar los mecanismos de coordinación correctamente, no optimizar throughput/latencia.
- **Reproducibilidad:** el entorno de pruebas debe poder levantarse localmente sin infraestructura externa (cluster de un solo host, múltiples procesos).
- **Tolerancia a fallos verificable:** el sistema debe seguir funcionando (con degradación explicable) si un broker no-controller cae.

Requisitos técnicos / de entorno

- Java 21, Maven como gestor de build (`pom.xml` configurado con `maven-shade-plugin`).
- Apache ZooKeeper 3.8.1 como servicio de coordinación (dependencia declarada; instancia de desarrollo corriendo en Docker).
- Control de versiones en Git, repositorio en GitHub.
- Docker, para levantar ZooKeeper local sin instalación manual.

4. Metodología y roles

Metodología: Kanban (exigida por la cátedra).

Al ser un proyecto individual, no hay división de roles entre personas — hay división de **roles funcionales** que un mismo integrante asume secuencialmente en cada tarjeta del tablero, algo que Kanban soporta naturalmente al no exigir roles fijos tipo Scrum (Product Owner / Scrum Master / Dev Team).

Rol funcional	Responsabilidad	Cuándo se activa
Analista de requisitos	Traduce cada etapa del artículo guía en tareas concretas y criterios de aceptación.	Al mover una tarjeta a "To Do"
Arquitecto / Diseñador	Decide estructura de clases, resuelve conflictos de diseño (ej. qué nodos de ZooKeeper deben ser persistentes vs. efímeros).	Al iniciar cada tarjeta
Desarrollador	Implementa el código.	"In Progress"
Tester / QA	Verifica en vivo contra una instancia real de ZooKeeper (Docker), no solo compilación.	"In Review"
Documentador	Registra decisiones y actualiza el repositorio.	Antes de mover a "Done"

Tablero Kanban propuesto (columnas): `Backlog` → `To Do` → `In Progress` → `In Review / Testing` → `Done` . Cada tarjeta corresponde a una etapa del artículo guía (o una subdivisión de esta). Se aplica un límite de trabajo en progreso (WIP = 1) dado que es un solo integrante — evita el error típico de avanzar una etapa nueva con la anterior a medio verificar (en este proyecto ya se dio un caso concreto: código de la etapa de elección de controller quedó mezclado dentro del archivo del cliente de ZooKeeper por trabajar dos frentes a la vez).

Fuente del proyecto guía: serie "Build Your Own Kafka" (buildthingsuseful, Medium), etapas 3 a 8.